Google Cloud Workshop

Gemini Enterprise Agent Platform Tour

Build, Deploy, Govern, Evaluate, and Optimize AI Agents

ADK Agents • MCP Servers • Agent Gateway • Agent Registry • Model Armor • Evaluation Pipeline

https://github.com/jswortz/geap-tour

Google Cloud

Table of Contents

Four sessions — from building agents to securing them in production

1

AI Gateway / MCP Gateway

Multi-agent architecture • MCP tool servers • ADK agents • Deployment • Workload identity

2

AI Gateway — Governance & Eval

Agent Gateway (dual mode) • Three-tier evaluation • Observability • GEPA optimization

3

Agent Registry

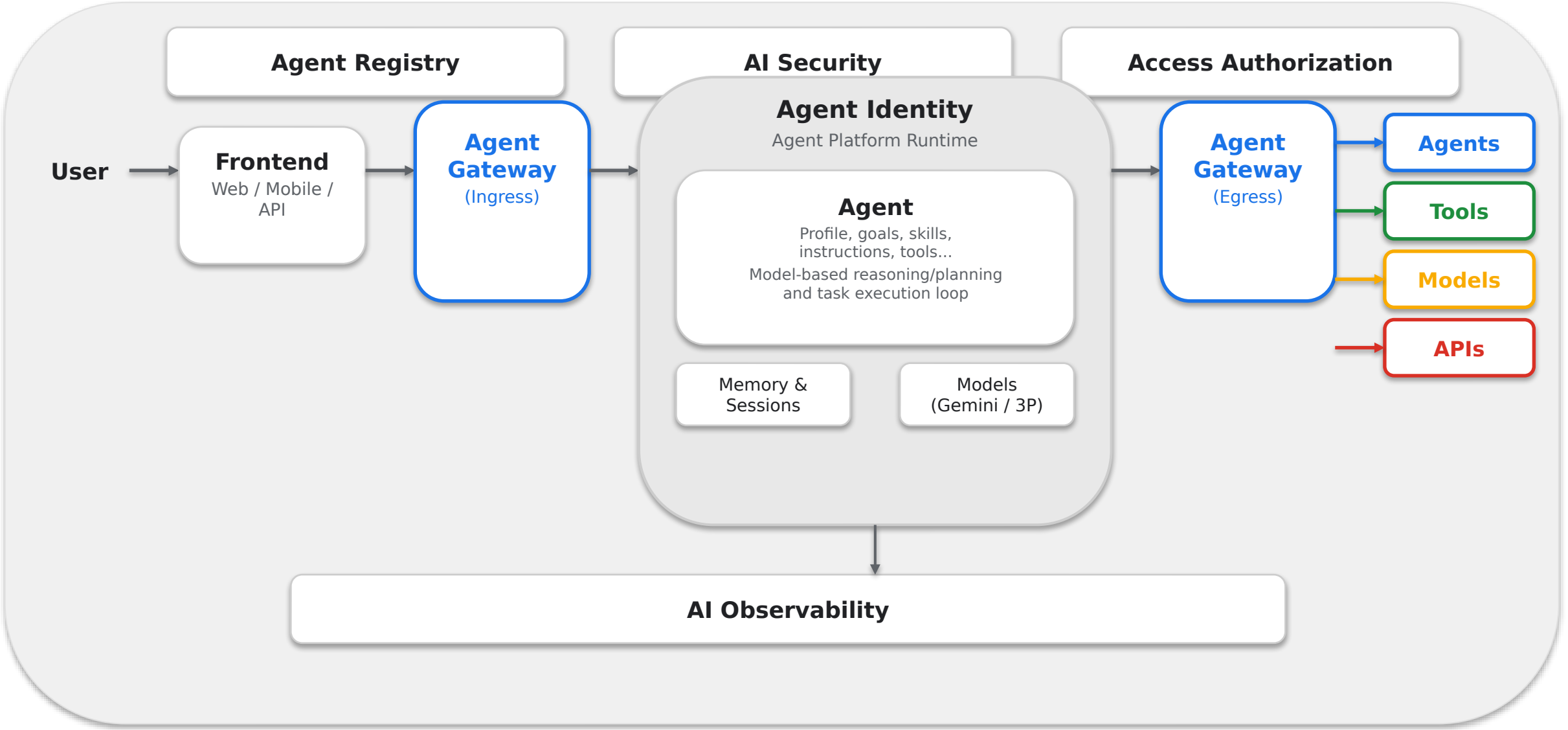
Agent registration & discovery • Toolspec association • Lifecycle governance

4

Model Security / Model Armor

Input/output screening • Prompt injection detection • Guardrail callbacks

Platform Architecture



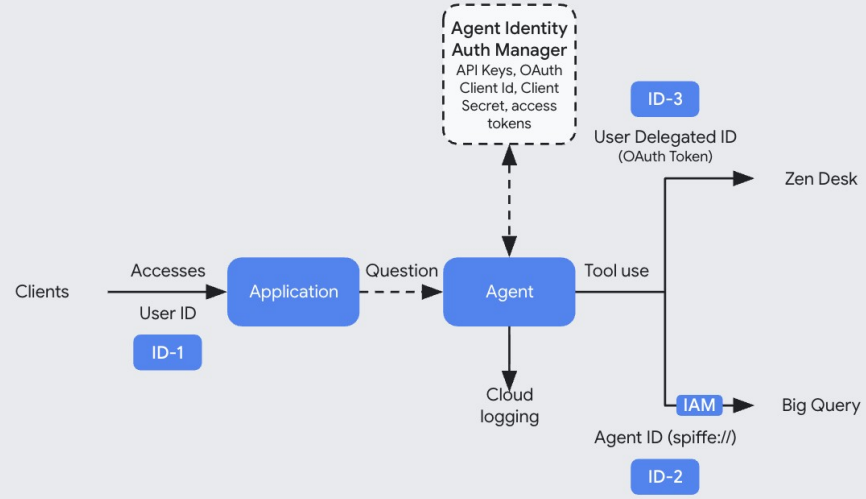
Agent Identity Model

Identities in Agentic Apps

Identity type	Purpose	Issuing system
ID-1 User Identity	Identity used by the user to access agent or SaaS (e.g. Gemini Enterprise)	Human IdP, e.g., Entra, Cloud Identity, Auth0, when user signs up
(ID-2) Agent Identity (Using its own authority)	Identity used by the agent to access resources under its own authority	GCP, when agent is created
(ID-3) Agent Identity (Using delegated authority from end-user)	Identity used by the agent to access resources using user's authority	OAuth server (1P or 3P), when user delegates access using OAuth dance

Example Scenario

User logs in to customer support application **(using user identity -ID1)**. User submits questions to an Agent that retrieves data from BQ **(using Agent Identity - ID2)** and Zendesk **(using user's delegated identity -ID3)** to answer the user's questions



Session 1

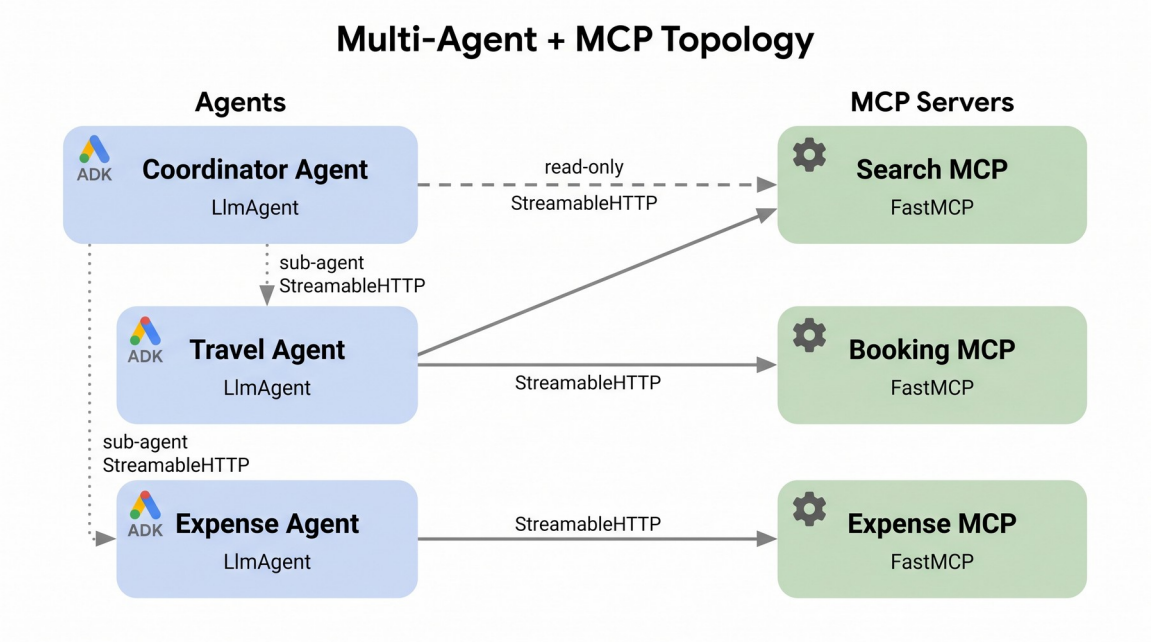
AI Gateway / MCP Gateway

Build, connect, and deploy agents with MCP tool servers

Multi-agent architecture with MCP connectivity
FastMCP tool servers on Cloud Run
ADK agents deployed to Agent Runtime
SPIFFE-based workload identity

https://github.com/jswortz/geap-tour

Multi-Agent Topology



Three ADK Agents

Coordinator Agent — routes requests to specialists

Travel Agent — searches flights/hotels, makes bookings

Expense Agent — submits expenses, enforces policy limits

Three MCP Servers

Search MCP — shared by Travel + Coordinator

Booking MCP — flight/hotel reservations

Expense MCP — expense submission + policy checks

MCP Server Development

FastMCP Pattern

```
from fastmcp import FastMCP

mcp = FastMCP("search-mcp")

@mcp.tool()
def search_flights(
    origin: str,
    destination: str,
    date: str | None = None
) -> list[dict]:
    """Search available flights."""
    return flight_db.search(
        origin, destination, date
    )
```

StreamableHTTP Transport

MCP servers run as standard HTTP services on Cloud Run with automatic scaling

Tool Discovery

Agents dynamically discover available tools via MCP protocol introspection

1-to-Many Topology

Multiple agents can share the same MCP server — no duplication needed

https://github.com/jswortz/geap-tour/src/mcp_servers/search/server.py

ADK Agent Architecture

Agent Definition

```
      from google.adk.agents import LlmAgent
from google.adk.integrations\
  .agent_registry import AgentRegistry

registry = AgentRegistry(
  project_id=PROJECT, location="global"
)

travel_agent = LlmAgent(
  name="travel_agent",
  model="gemini-2.5-flash",
  instruction="""You are a travel
  assistant...""",
  tools=[
    registry.get_mcp_toolset(
      SEARCH_MCP_SERVER
    )
  ]
)
```

Coordinator Pattern

Root agent uses sub_agents=[travel, expense] to delegate by intent

Session Management

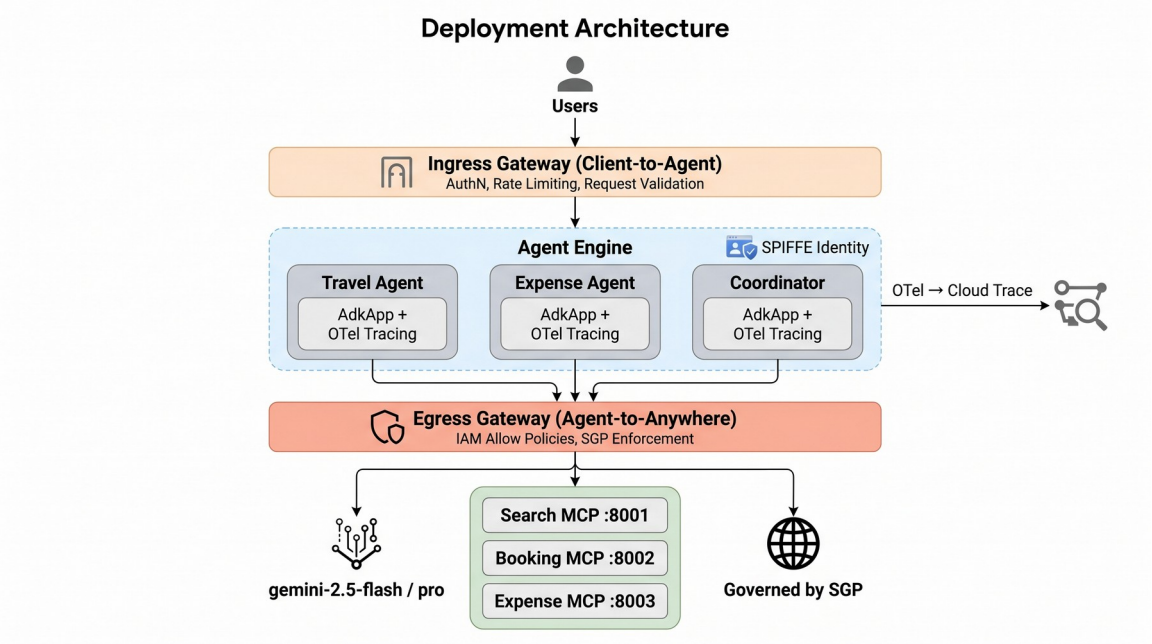
Each user gets isolated sessions with conversation history and state

OTel Tracing

Every agent call produces OpenTelemetry spans — exported to Cloud Trace automatically

https://github.com/jswortz/geap-tour/src/agents/coordinator_agent.py

Deployment Architecture



Cloud Run — MCP Servers

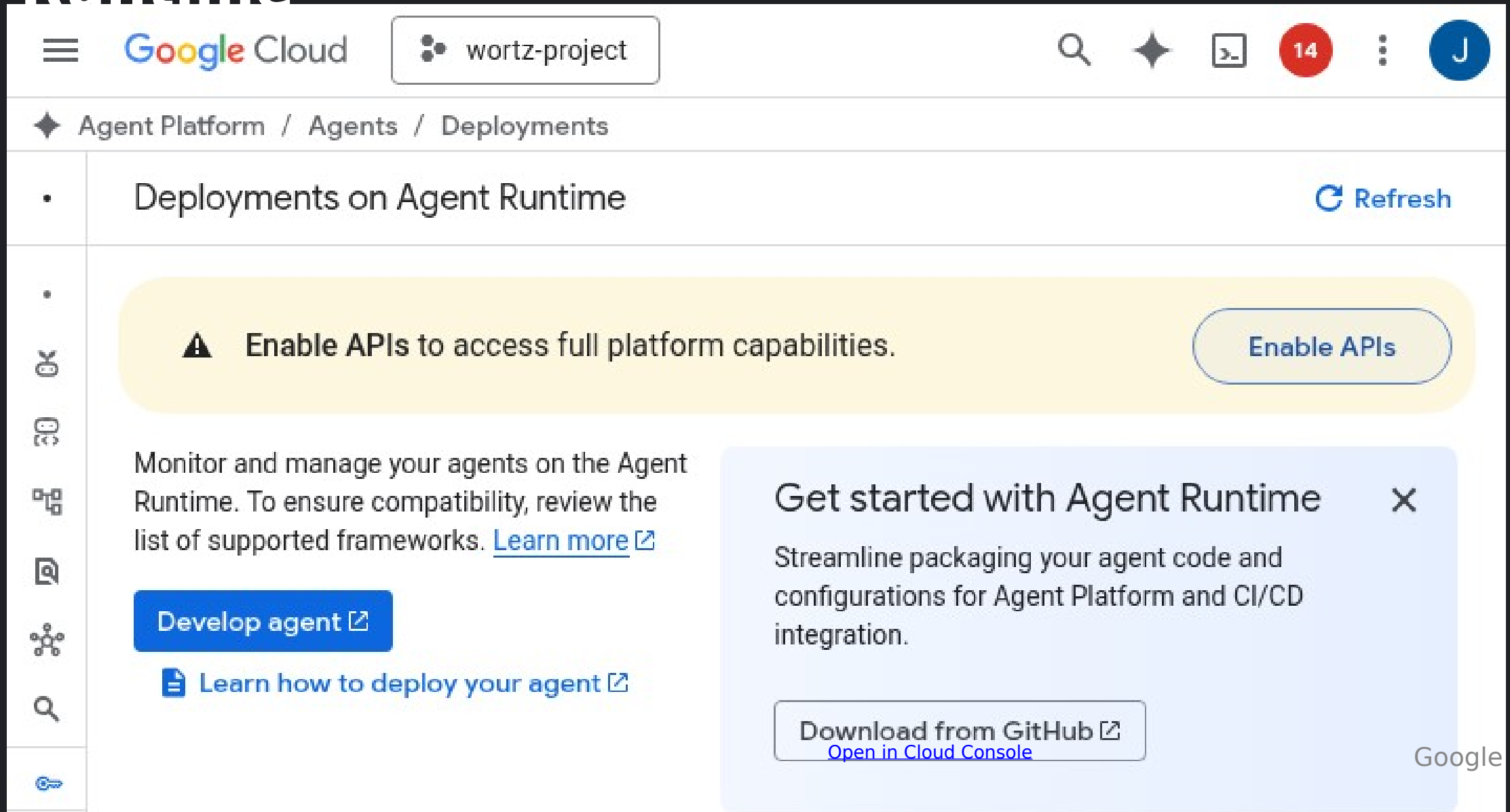
StreamableHTTP transport, auto-scaling, IAM-secured endpoints

Agent Runtime — ADK Agents

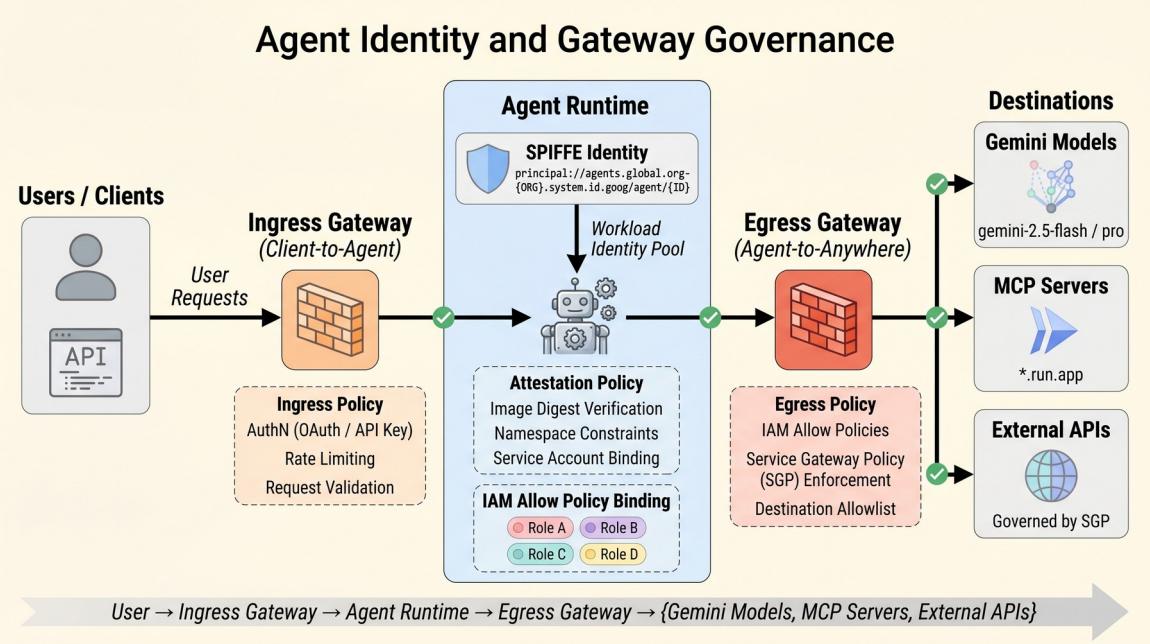
Vertex AI Agent Engine with built-in session management and memory

One command deploy: bash scripts/deploy_all.sh
Deploys 3 MCP servers + coordinator agent in sequence

Console: Deployments on Agent Runtime



Agent Identity — SPIFFE



Workload Identity Federation

Each agent gets a SPIFFE ID at deployment

Attestation policies verify agent identity at runtime

Agent Gateway enforces identity at network boundary

Type	Purpose
ID-1: User	Human accessing the app
ID-2: Agent	Agent's own authority
ID-3: Delegated	Agent on behalf of user

Workload Identity — Setup & Verification

Create Identity Pool & Provider

```
# Create workload identity pool
gcloud iam workload-identity-pools \
  create geap-agent-pool \
  --location="global" \
  --display-name="GEAP Agent Pool"

# Create OIDC provider
gcloud iam workload-identity-pools \
  providers create-oidc agent-provider \
  --workload-identity-pool=geap-agent-pool \
  --issuer-uri="https://accounts.google.com" \
  --location="global"
```

Verify token exchange: IAM & Admin > Workload Identity Federation

Pool = Trust Boundary

Each pool groups external identities from one trust domain — agents get their own pool

Provider = Auth Source

OIDC or SAML provider that issues tokens to agents — verified at runtime by GCP

Token Exchange

External token → STS → short-lived GCP credential — no service account keys needed

https://github.com/jswortz/geap-tour/scripts/setup_agent_identity.sh

Session 2

AI Gateway / MCP Gateway (continued)

Gateway, evaluation, observability, and optimization

Agent Gateway for ingress/egress control
Three-tier evaluation pipeline
Online monitors and failure cluster analysis
Agent optimization with GEPA algorithm

https://github.com/jswortz/geap-tour

Agent Gateway — Dual Mode

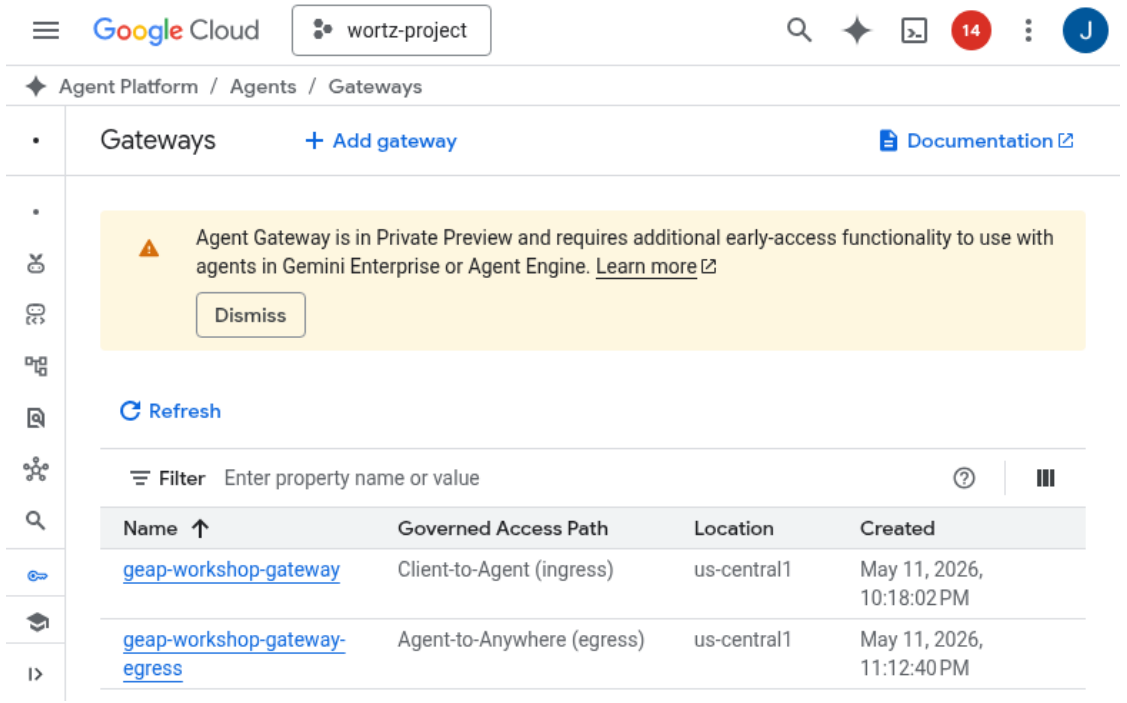
Ingress + Egress

Mode	Gateway	Controls
Client-to-Agent	geap-workshop-gateway	Inbound user requests
Agent-to-Anywhere	geap-workshop-gateway-egress	Gemini model calls, MCP tools, external APIs

With egress gateway, ALL outbound traffic — including Gemini model calls — routes through governance (IAM Allow + SGP + Model Armor)

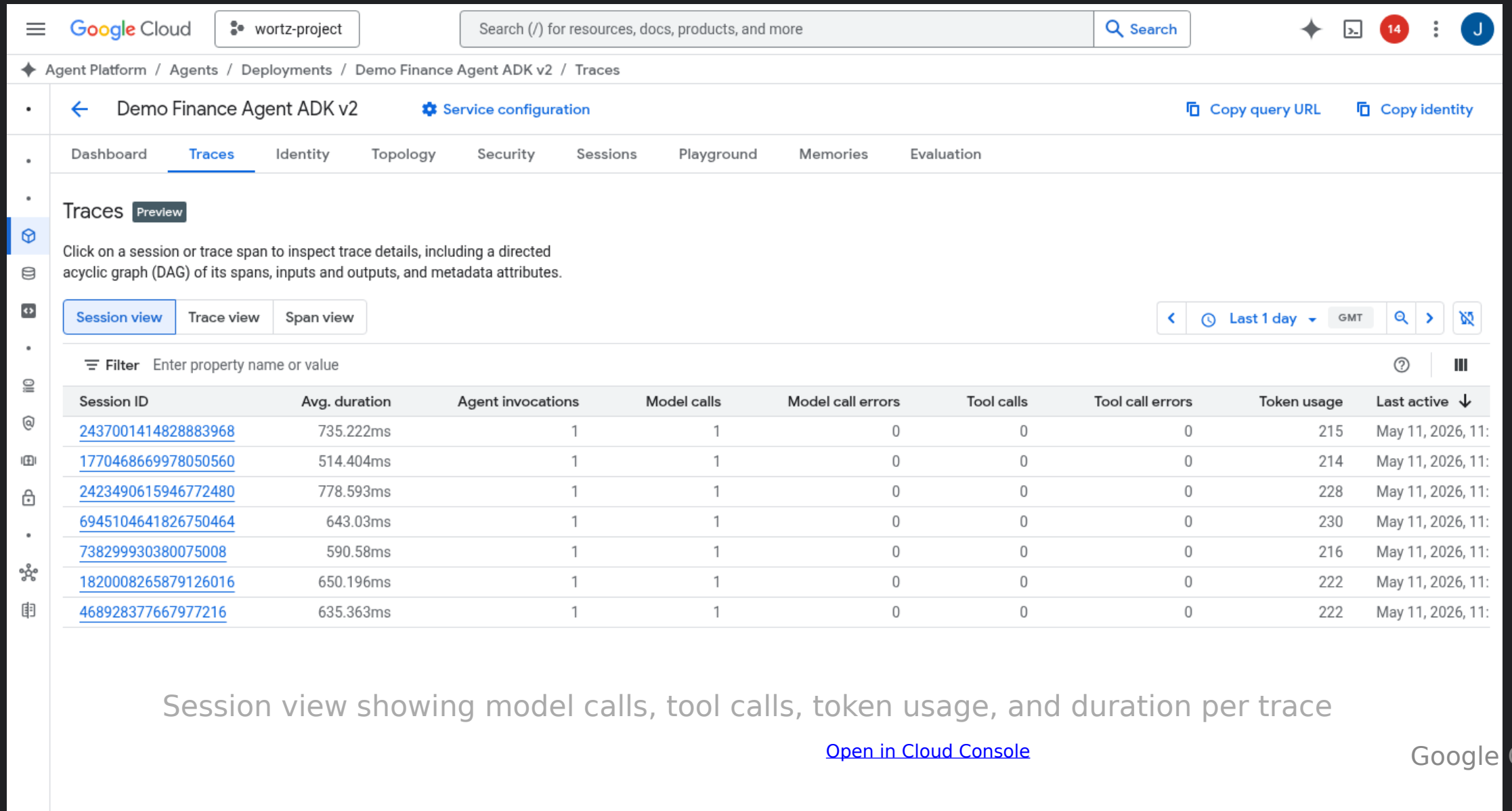
```
                        config = {
  "agent_gateway_config": {
    "client_to_agent_config": {"agent_gateway": INGRESS},
    "agent_to_anywhere_config": {"agent_gateway":
EGRESS},
  },
  "identity_type": "AGENT_IDENTITY",
}
```

Requires Private Preview — gateway attachment via REST API PATCH



https://github.com/jswortz/geap-tour/scripts/setup_agent_gateway.sh

Console: Agent Traces



Session view showing model calls, tool calls, token usage, and duration per trace

Open in Cloud Console

Governance — Three Policy Layers

Layer	Type	Enforces
IAM Allow	Static (CEL rules)	Which MCP servers an agent can call
SGP	Runtime (natural language)	Business rules — e.g. "Booking < $5,000"
Model Armor	Content screening	Prompt injection, jailbreak, PII, harmful content

Layered defense: IAM Allow restricts where agents connect, SGP restricts what agents do, Model Armor restricts how content is screened

IAM Allow Policy (CEL)

```
          resource.type == "networkservices"
&& resource.service in [
  "search-mcp",
  "booking-mcp",
  "expense-mcp"
]
```

Semantic Governance Policy

```
              name: "geap-expense-limit"
constraint: "Disallow expense
  submissions exceeding $200
  for meals, $500 for
  entertainment"
verdict: DENY
```

https://github.com/jswortz/geap-tour/scripts/setup_governance_policies.sh

Delegating Authorization to the Gateway

AuthzExtensions wire governance into the request path

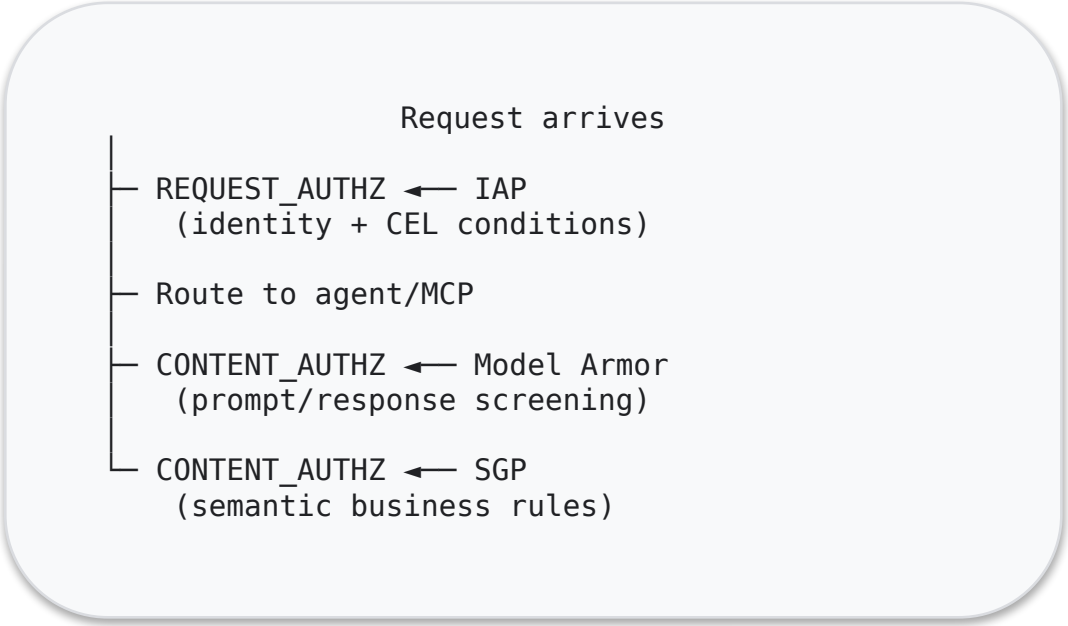
Extension	Profile	What It Sees	Service
IAP	REQUEST_AUTHZ	Headers only	iap.googleapis.com
Model Armor	CONTENT_AUTHZ	Full request + response body	modelarmor.REGION.rep.googleapis.com
SGP	CONTENT_AUTHZ	Full request + response body	SGP engine (VPC DNS)
Custom	Either	Configurable	Your FQDN (VPC)

Max 4 policies per gateway | REQUEST_AUTHZ runs first | CONTENT_AUTHZ sees bodies

MCP Tool-Level Policies (inline, no extension)

```
                    httpRules:
  - to:
      operations:
        - mcp:
            baseProtocolMethodsOption:
              MATCH_BASE_PROTOCOL_METHODS
            methods:
              - name: "tools/call"
                params: ["search_flights"]
```

Request Flow Through Gateway

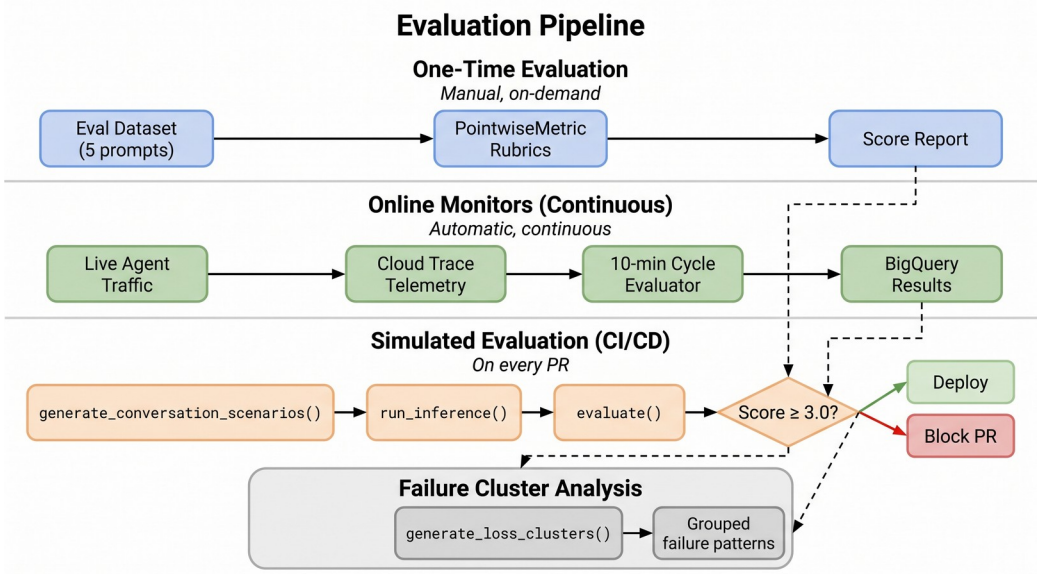


Setup (REST API)

```
                    # 1. Create extension
POST networkservices/v1beta1/
  .../authzExtensions
  {service, failOpen, timeout}

# 2. Create policy
POST networksecurity/v1beta1/
  .../authzPolicies
  {target, policyProfile, action}
```

Three-Tier Evaluation Pipeline



One-Time Eval

Manual, on-demand evaluation with PointwiseMetric rubrics against a curated dataset

Online Monitors

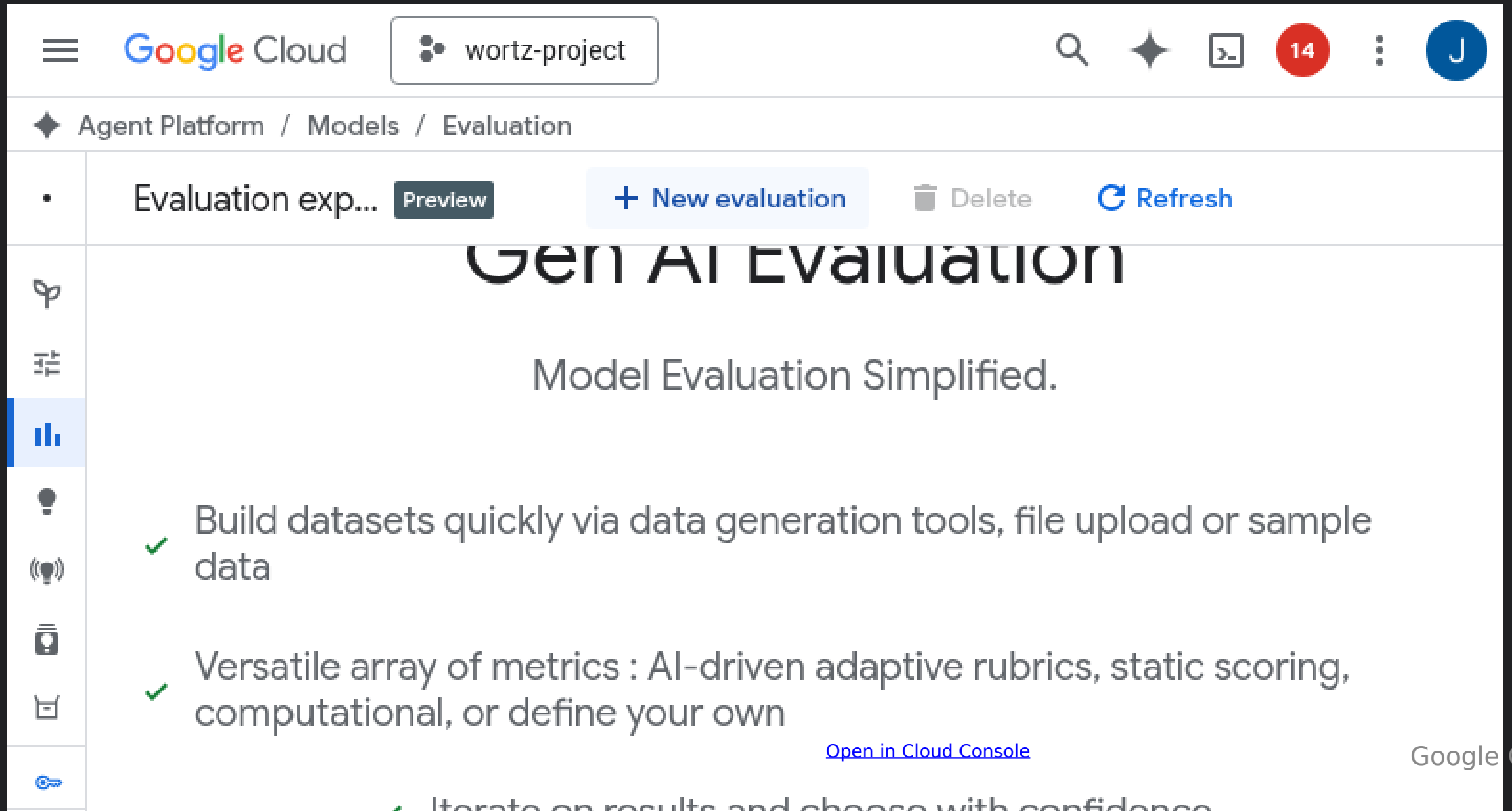
Continuous evaluation of live traffic via Cloud Trace telemetry on 10-min cycles

Simulated (CI/CD)

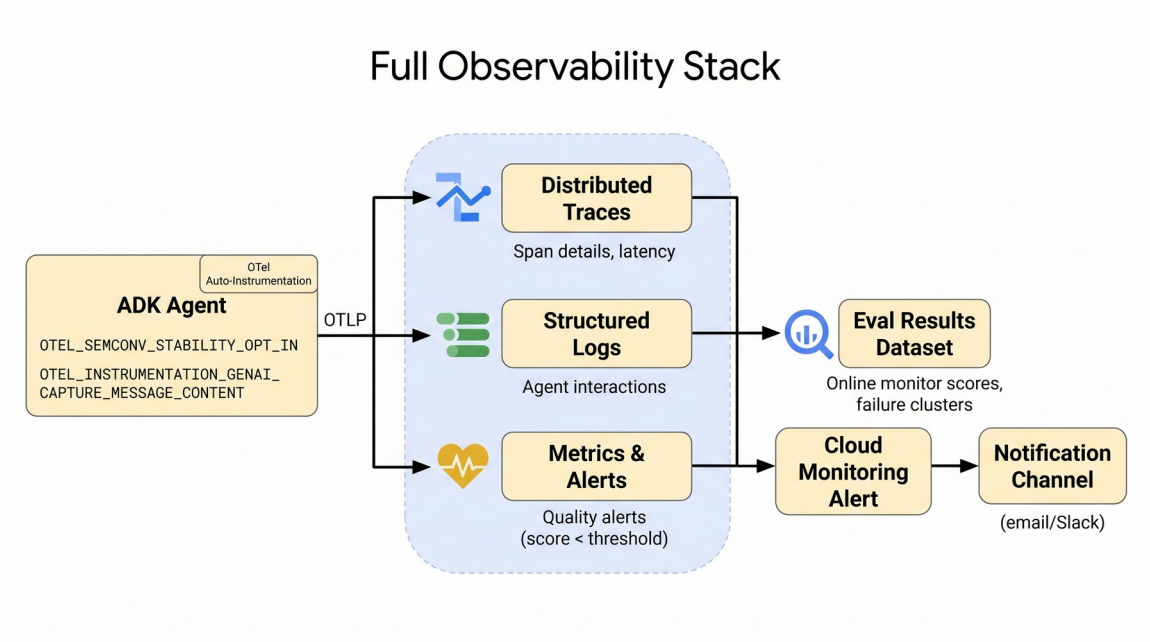
Automated eval gate on PRs — score ≥ 3.0 to merge, blocks otherwise

https://github.com/jswortz/geap-tour/src/eval/

Console: Evaluation Experiments



Observability Stack



End-to-End Tracing

Agent Call → OTel Spans → Cloud Trace → BigQuery

Data Pipeline

Cloud Trace captures every agent + tool call

Log sink exports structured data to BigQuery

Cloud Monitoring alerts on anomalies

Failure cluster analysis groups error patterns

Failure Clusters & Quality Alerts

Automated Error Analysis

```
clusters = client.evals
  .generate_loss_clusters(
    src=eval_result_name
  )

for cluster in clusters:
  print(cluster.title)
  print(cluster.description)
  print(f"Samples: {cluster.sample_count}")
  print(f"Avg score: {cluster.avg_score}")
```

Groups similar failures by semantic similarity — surface systemic issues instead of debugging one trace at a time

Failure Clusters

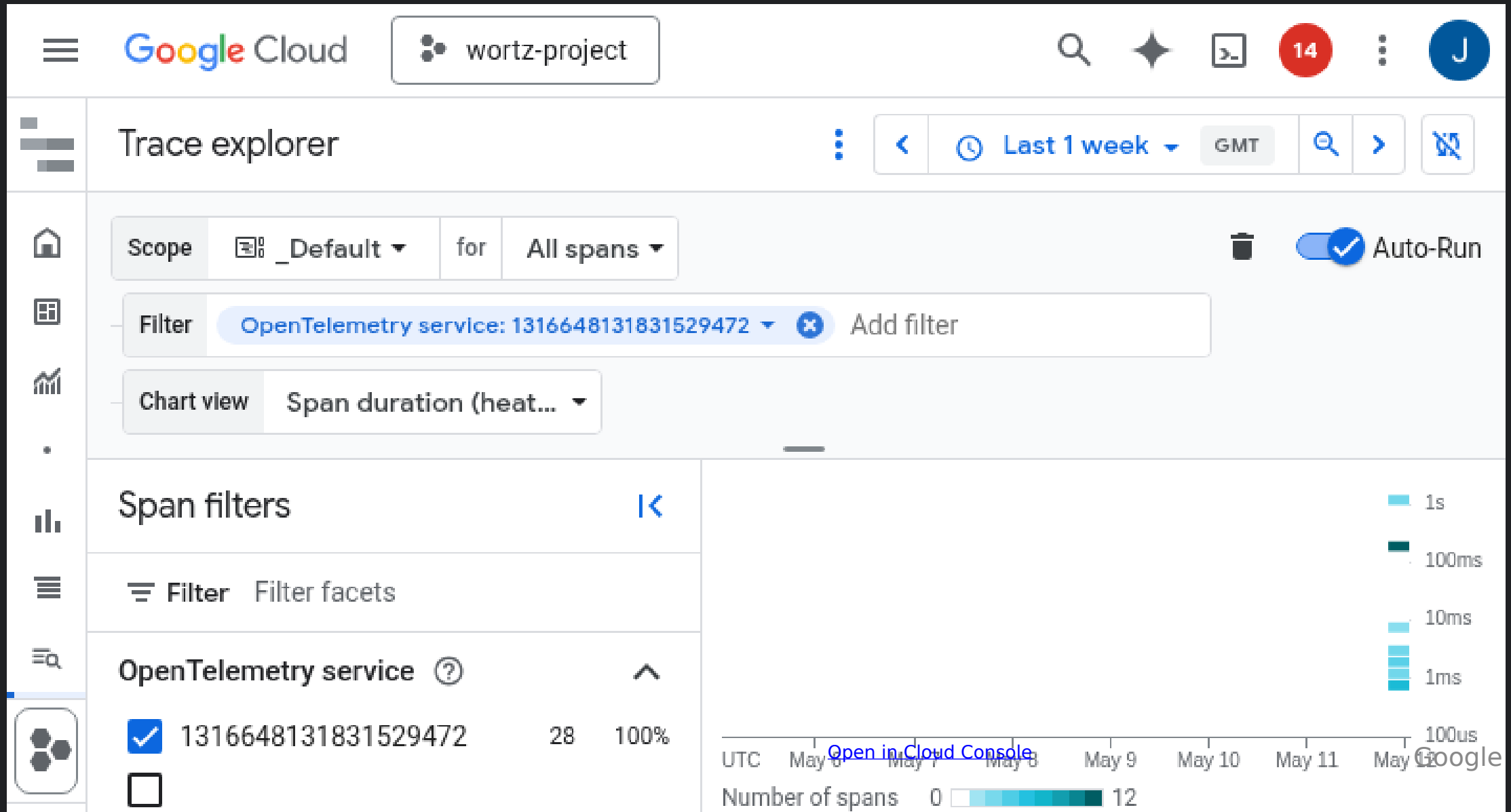
Semantic grouping of eval failures into themes: tool misuse, routing errors, policy violations. Each cluster includes sample count, avg score, and representative examples.

Quality Alerts

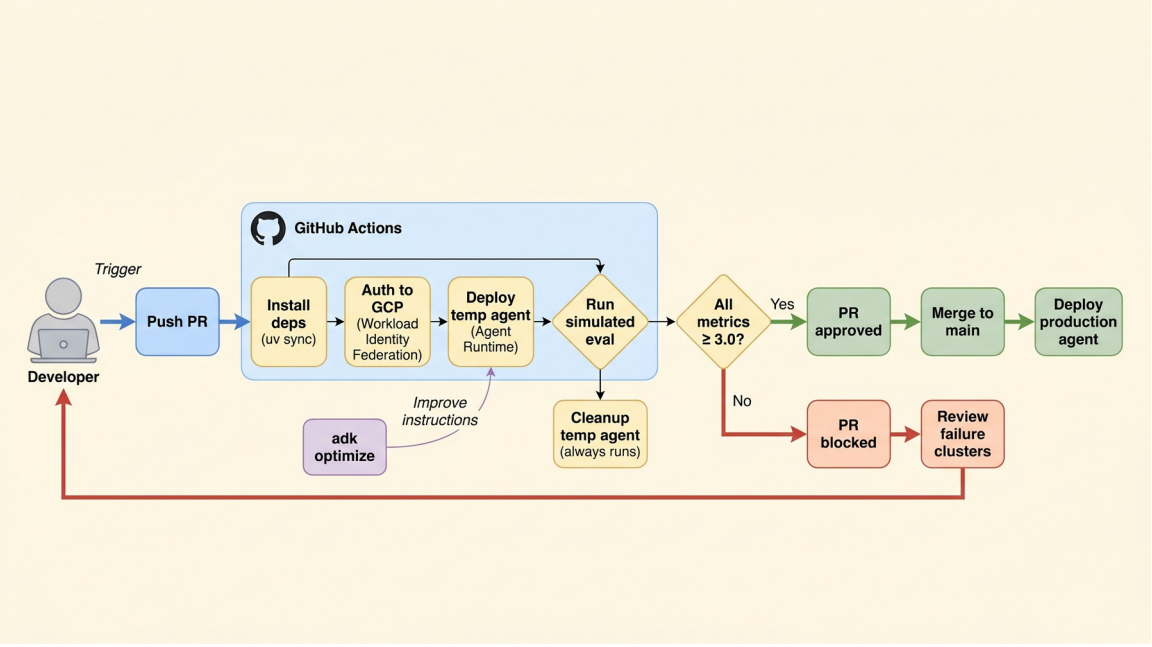
Cloud Monitoring alert policies fire when eval scores drop below threshold. 10-minute aggregation window with custom metric: agent_eval/{metric_name}.

https://github.com/jswortz/geap-tour/src/eval/quality_alerts.py

Console: Cloud Trace



CI/CD — Simulated Eval Gate



GitHub Actions Workflow

PR Opened → Generate Scenarios → Run Inference → Evaluate → Score ≥ 3.0?

✅ Pass — Merge allowed
❌ Fail — PR blocked with failure report

Agent Optimization — GEPA Algorithm

Gemini Evolutionary Prompt Algorithm

Generate prompt variants using LLM mutation

Evaluate each variant against test scenarios

Select top performers based on eval scores

Evolve over multiple generations

Result: Automatically discovers better agent instructions without manual prompt engineering

Configuration

```
optimizer = AgentOptimizer(
  agent=coordinator_agent,
  eval_dataset=eval_data,
  metrics=[
    "response_quality",
    "tool_selection",
    "safety"
  ],
  generations=5,
  population_size=4
)
```

https://github.com/jswortz/geap-tour/src/optimize/run_optimize.py

Multi-Model Router

Complexity-Based Routing Table

Complexity	Model	Score Range	Use Case
Low	Flash Lite	< 0.35	Simple lookups
Medium	Gemini Flash	0.35 – 0.64	Multi-step queries
High	Claude Opus	≥ 0.65	Deep analysis

Cost savings: 60-80% by routing simple queries to lightweight models

before_agent_callback

```
     async def complexity_router_callback(
    callback_context=None, **kwargs
):
    user_message = ""
    if callback_context.user_content:
        for part in
callback_context.user_content.parts:
            if part.text:
                user_message += part.text

    result = await
classify_complexity(user_message)
    ctx = callback_context.state
    ctx["complexity_level"] = result.level
    ctx["complexity_score"] = result.score
    return None
```

https://github.com/jswortz/geap-tour/src/router/agents.py

Memory Bank & User Namespaces

```
          coordinator_agent = LlmAgent(
    model=AGENT_MODEL,
    name="coordinator_agent",
    tools=[
        get_mcp_tools(SEARCH_MCP_SERVER),
        # Retrieves relevant memories at
        # turn start, injects into system
        # instruction
        PreloadMemoryTool(),
    ],
    after_agent_callback=
        save_memories_callback,
)

async def save_memories_callback(ctx):
    await ctx.add_events_to_memory(
        events=ctx.session.events,
        custom_metadata={"wait_for_completion":
True}
    )
    return None
```

Memory Recall

PreloadMemoryTool retrieves relevant memories at turn start and injects them into the system instruction — the agent has user context before it responds

User Namespaces

Memories are scoped to {user_id, app_name} — each user gets an isolated memory space with no cross-contamination

https://github.com/jswortz/geap-tour/src/agents/coordinator_agent.py

Session 3

Agent Registry

Agent registration, discovery, and governance

https://github.com/jswortz/geap-tour

Google Cloud

Agent Registry & Discovery

Registry Capabilities

Register agents with metadata, toolspecs, and versioning

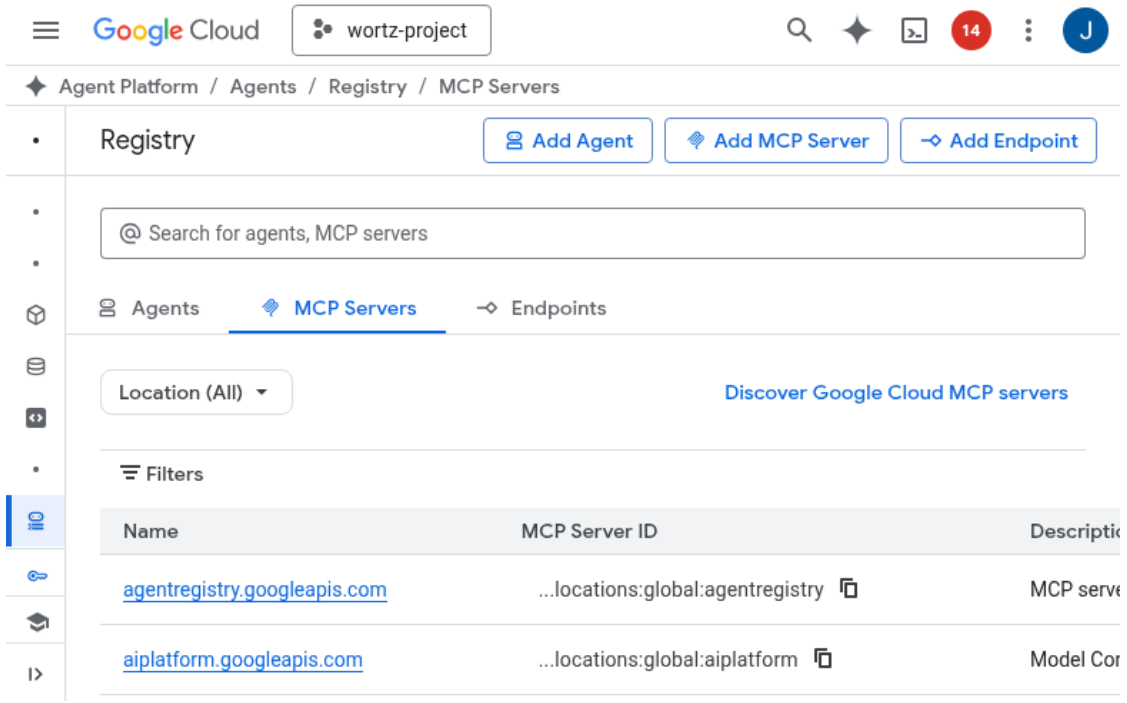
Discover available agents and their capabilities

Associate MCP tool specifications with agents

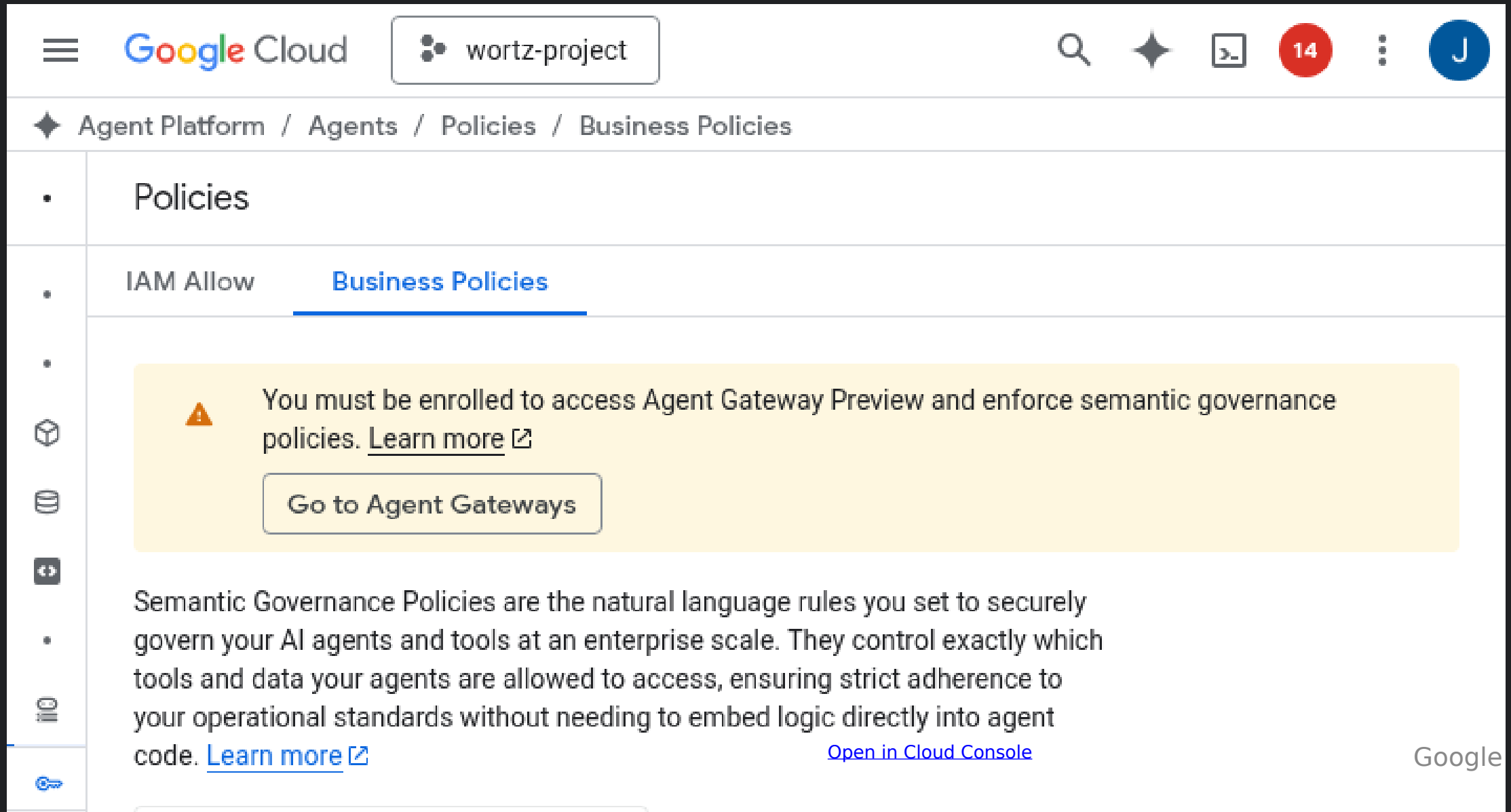
Govern agent lifecycle and access control

Toolspec Association

```
    gcloud agent-platform agent-registry \
toolspecs associate \
--agent=geap-coordinator \
--toolspec=search-mcp-spec.yaml
```



Console: Business Policies



Session 4

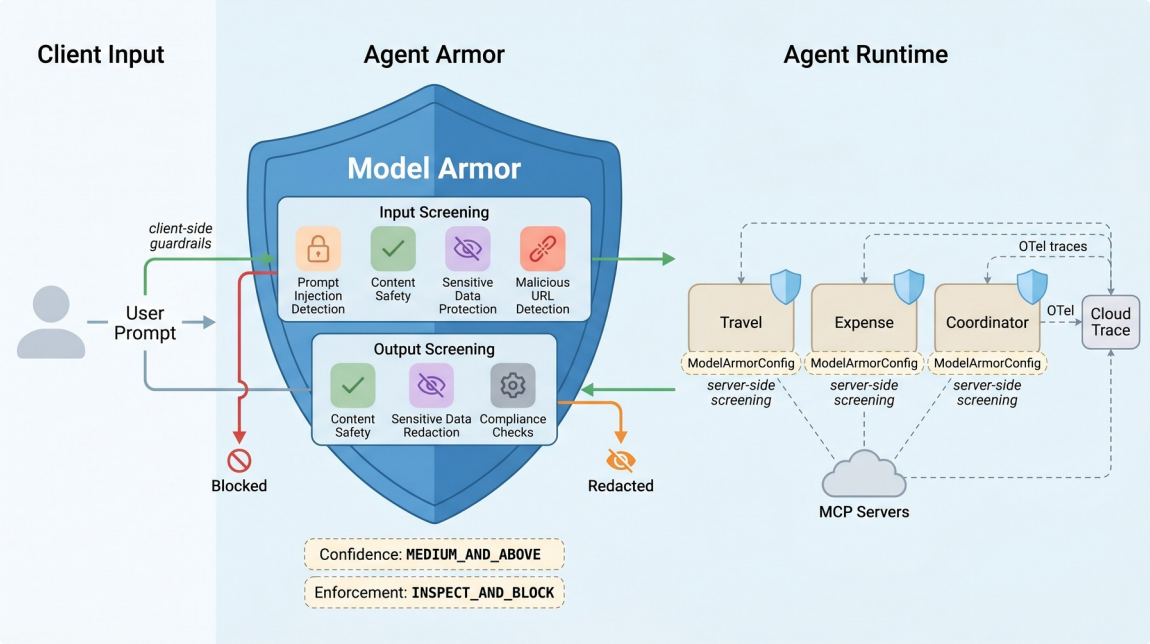
Model Security / Model Armor

Input/output screening, guardrails, and content safety

https://github.com/jswortz/geap-tour

Google Cloud

Model Armor — Content Safety Integration



Input Screening

Prompt injection detection, jailbreak prevention, PII filtering before model call

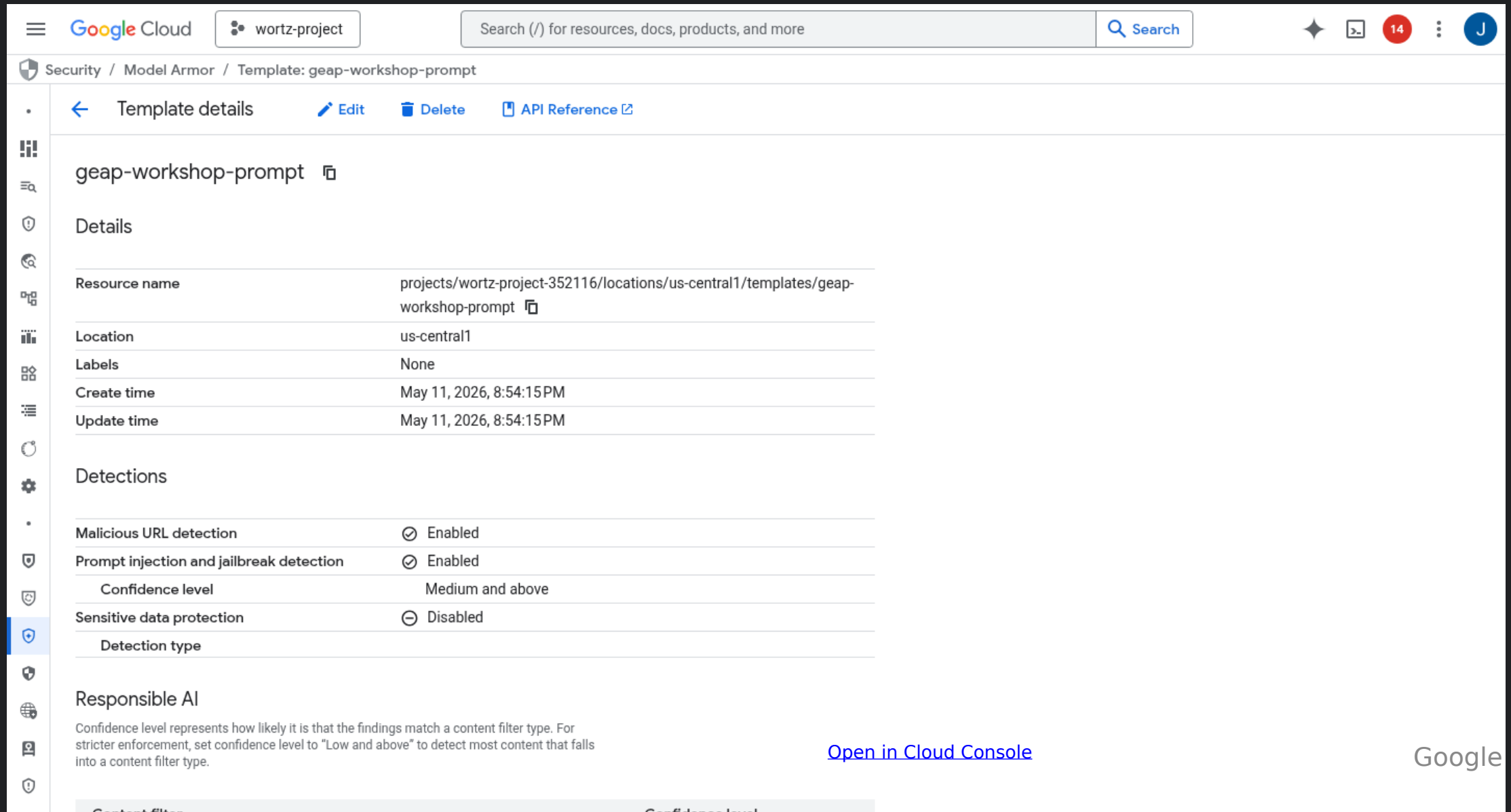
Output Screening

Harmful content filtering, hallucination flags, safety score thresholds

ADK Guardrail Callbacks

before_model_callback and after_model_callback hooks integrated into agent lifecycle

Console: Model Armor



Platform Summary

Build
- ADK agents with LlmAgent
- FastMCP tool servers
- Memory Bank + Router

Deploy
- Cloud Run (MCP servers)
- Agent Runtime (agents)
- One-command deployment

Govern
- SPIFFE identity
- Agent Gateway (dual)
- Agent Registry

Secure
- Model Armor templates
- Input/output screening
- Guardrail callbacks

Evaluate
- One-time eval
- Failure clusters
- CI/CD eval gate

Optimize
- GEPA algorithm
- Multi-model routing
- OTel observability

Hands-On: Two SDK-First Notebooks

Learn the platform by running L1-SDK code — build first, then measure quality

① Build → Deploy → Register

src/deploy/demo/platform_sdk_demo.ipynb — MCP tools → ADK agents → run → deploy to Agent Engine → register to Gemini Enterprise → multi-model routing & cost. Mutating steps are guarded (GEAP_RUN_DEPLOY / GEAP_PUBLISH).

② Evaluate — the Quality Flywheel

src/eval/demo/evaluation_sdk_demo.ipynb — every phase calls client.evals.* / vertexai.types.* inline: metrics, rapid/regression/simulated/offline scoring, online monitors, optimization, and quality alerts.

Both are flat, SDK-first, and validated end-to-end via nbconvert; custom/infra code is marked 🔧.

https://github.com/jswortz/geap-tour/src/

Get Started

Workshop repo: https://github.com/jswortz/geap-tour

Workshop guide: docs/workshop_guide.md

Build notebook: src/deploy/demo/platform_sdk_demo.ipynb

Eval notebook: src/eval/demo/evaluation_sdk_demo.ipynb

Agent Development Kit: google.github.io/adk-docs

MCP Protocol: modelcontextprotocol.io

Agent Platform: cloud.google.com/agent-platform

Thank you!